TFG

Estructura general:

Pagína principal: bienvenida a la página de que va, como funciona etc, sección de libros mas leidos o algo asi

Pagína de los clubes de lectura: Clubs disponibles, opción de unirse(o solicitar unirse) a uno. Los administradores ponen un libro semanal y manejan la creación de chats.

Página de estanterias: los usarios podrán hacer tantas estanterías como quieran, mover libros entre estanterías y un libro podrá estar en varias estanterías a la vez.

Página del perfil del usuario: podrá cambiar su foto de perfil, información y etc.

1) Usuarios y perfiles

users

- id (PK)
- email (UNIQUE)
- password_hash
- roles (json) → ej: ["ROLE_USER"]
- created_at, updated_at
- is_active (bool)

user_profile (1:1 con users)

- user_id (PK y FK a users.id)
- display_name
- bio
- avatar_url (o avatar_path)
- location (opcional)
- updated_at

Tabla books (opción A)

books

Campos recomendados:

- id (PK, autoincrement)

- external_source (varchar, nullable) → ej: "google_books", "openlibrary"
- external_id (varchar, nullable) → id de la fuente
- title (varchar, NOT NULL)
- authors (json o text, nullable) → (en Symfony puedes mapearlo como json)
- isbn10 (varchar(10), nullable)
- isbn13 (varchar(13), nullable)
- cover_url (text, nullable)
- description (text, nullable)
- publisher (varchar, nullable)
- published_date (date o varchar, nullable) → muchas APIs devuelven solo año/mes
- language (varchar(8), nullable) → "es", "en", etc.
- page_count (int, nullable)
- categories (json o text, nullable)
- created_at (datetime)
- updated_at (datetime)

Para las búsquedas:

- • `UNIQUE(external_source, external_id)` (cuando existan)
- • `UNIQUE(isbn13)` (nullable)
- • `UNIQUE(isbn10)` (nullable)
- • `INDEX(title)`
- • (opcional) `FULLTEXT(title, description, authors)` si usas MySQL y quieres búsqueda más pro

CACHÉ del buscador para no recargar la API:

💡 **Combinar ambos** (esto queda 🔥 en la defensa):

Flujo ideal:

1. React mira en localStorage
2. Si existe → usa eso
3. Si no:
    - llama a Symfony

    - Symfony:
        - busca en books
        - si no hay → llama a Google Books
        - guarda en books
4. React guarda el resultado en localStorage

👉 Resultado:

- UX rápida
- Backend limpio
- Nada se rompe si alguien borra el navegador

Tabla para el cache:

book_search_cache (opcional)

- id
- query (varchar)
- source (varchar) → "google_books"
- response_json (json/text)
- created_at
- expires_at (datetime)

MONTAJE:

Cómo encaja con el resto

- Estanterías (shelf_books) apuntan a book_id.
- Libro mensual del club (club_monthly_books) apunta a book_id.
- Buscador: buscas en tu tabla books; si no hay suficientes, consultas API, guardas en books, devuelves resultados.

TABLAS DEL CLUB DE LECTURA

clubs

- id (PK)
- name
- description
- owner_id (FK users.id) → creador (puede ser admin principal)
- visibility (enum: public, private) (si lo quieres a futuro)
- created_at, updated_at

club_members

Tabla puente N:M (un usuario puede estar en muchos clubs, un club tiene muchos miembros).

- club_id (FK)
- user_id (FK)
- role (enum: member, admin) → **importante**: “administrador del club”
- joined_at
- **PK compuesta** (club_id, user_id)

Con esto controlas permisos: solo admin puede crear chats, fijar libro mensual, etc.

club_join_requests

(solicitudes pendientes)

- id
- club_id (FK)
- user_id (FK) → quien solicita
- status ENUM: pending, approved, rejected (o solo pending y borras al resolver, como prefieras)
- requested_at
- resolved_at (nullable)
- resolved_by_user_id (FK users, nullable) → admin que acepta/rechaza
- **UNIQUE (club_id, user_id)** → una solicitud por persona/club

club_join_requests

(solicitudes pendientes)

- id
- club_id (FK)
- user_id (FK) → quien solicita
- status ENUM: pending, approved, rejected (o solo pending y borras al resolver, como prefieras)
- requested_at
- resolved_at (nullable)
- resolved_by_user_id (FK users, nullable) → admin que acepta/rechaza
- **UNIQUE (club_id, user_id)** → una solicitud por persona/club

4) Libro mensual del club (histórico)

En vez de guardar "un libro actual" en clubs, te recomiendo histórico por meses (queda muy bien en memoria del TFG).

club_monthly_books

- id (PK)
- club_id (FK)
- book_id (FK books.id)
- month (DATE o string YYYY-MM-01)
- set_by_user_id (FK users.id) → quién lo puso
- created_at
- UNIQUE (club_id, month) → evita 2 libros el mismo mes

5) Chats en directo dentro del club (varios chats)

club_chats

- **id**
- **club_id (FK)**
- **title**
- **created_by_user_id (FK)**
- **is_open (bool)**
- **created_at**

- **closed_at (nullable)**

club_chat_messages

- **id**
- **chat_id (FK)**
- **user_id (FK)**
- **content (text)**
- **created_at**
- **Índice recomendado: (chat_id, created_at)**

Qué hace React (fácil):

- **Cada 2–3 segundos llama a "traer mensajes nuevos" (polling).**

TABLAS DE LAS ESTANTERÍAS DEL USUARIO

shelves

- id (PK)
- user_id (FK users.id)
- name
- order_index (int) → para ordenar estanterías en UI
- created_at, updated_at

shelf_books

- **id**
- **shelf_id (FK)**
- **book_id (FK)**
- **order_index**
- **status (opcional)**
- **added_at**
- **UNIQUE (shelf_id, book_id) → evita duplicarlo dentro de la misma estantería, pero permite que esté en otras estanterías.**

Resumen de relaciones (para tu diagrama ER)

- users 1—1 user_profile

- users 1—N clubs (como owner)
- users N—M clubs vía club_members
- clubs 1—N club_chats
- club_chats 1—N club_chat_messages
- clubs 1—N club_monthly_books
- books 1—N club_monthly_books
- users 1—N shelves
- shelves 1—N shelf_books
- books 1—N shelf_books

Reglas de negocio que salen directas del modelo

- “Ver sin loguearse” → acceso público a clubs, books y a vista de estanterías si decides que sean privadas (normalmente sí: solo dueño).
- “No interactuar sin registrarse” → backend exige auth para:
    - unirse a club (club_members)
    - crear club (clubs)
    - crear chats (club_chats) **solo rol admin**
    - enviar mensajes (club_chat_messages)
    - crear estanterías y gestionar libros (shelves, shelf_books)
- “Admin del club crea tantos chats como quiera” → club_members.role = admin habilita.

+ Especifiaciones:

 Libros: opción A (tabla books propia) ✅

 Un mismo libro puede estar en varias estanterías ✅

 Clubs: pueden ser **públicos o privados** (privado = con **solicitud** tipo Instagram) ✅

 Chat: empezamos por la opción fácil (**REST + polling**) y si sobra tiempo, “tiempo real de verdad” ✅

ENDPOINTS MINIMOS:

Clubs

- GET /api/clubs (público: lista clubs visibles)
- POST /api/clubs (logueado)
- POST /api/clubs/{id}/join
    - si público: crea miembro
    - si privado: crea solicitud
- GET /api/clubs/{id}/requests (solo admin)
- POST /api/clubs/{id}/requests/{requestId}/approve (solo admin)
- POST /api/clubs/{id}/requests/{requestId}/reject (solo admin)

Chats (polling)

- POST /api/clubs/{id}/chats (solo admin)
- GET /api/chats/{chatId}/messages?after=timestamp (logueado y miembro)
- POST /api/chats/{chatId}/messages (logueado y miembro)

Orden de creación de tablas con Doctrine para no liarnos (Tabla User ya creada):

Orden recomendado (para no liarnos)

1. **Book** (base para buscador, estanterías y clubs)
2. **Shelf** + **ShelfBook** (estanterías del usuario)

3. **Club** + **ClubMember** + **ClubJoinRequest** (público/privado con solicitudes)
4. **ClubChat** + **ClubChatMessage** (chat fácil con polling)
5. (Opcional) **UserProfile** (foto/bio)

Creación identidad book -> poner createdAt y updatedAt automáticos

Cómo se divide correctamente tu proyecto (esto es CLAVE)

Tu TFG tiene **dos capas bien separadas**:

1⃣ Backend (Symfony – API REST)

Esto es lo que **ya estás haciendo ahora** y **NO es frontend**:

- Entidades (Book, Shelf, Club, Chat…)
- Base de datos
- Seguridad (quién puede hacer qué)
- Endpoints tipo:
  - GET /api/clubs
  - POST /api/clubs/{id}/join
  - GET /api/chats/{id}/messages

2⃣ Frontend (React)

Esto viene **después** y consiste en:

- Páginas (Home, Clubs, Estanterías, Perfil…)
- Formularios
- Botones
- Llamadas fetch / axios a la API Symfony
- Mostrar datos

👉 Paso 2 — API REST (LO QUE TOCA AHORA)

Crear **controladores Symfony** que devuelvan JSON:

Clubs

- listar clubs
- unirse
- solicitudes
- aprobar/rechazar

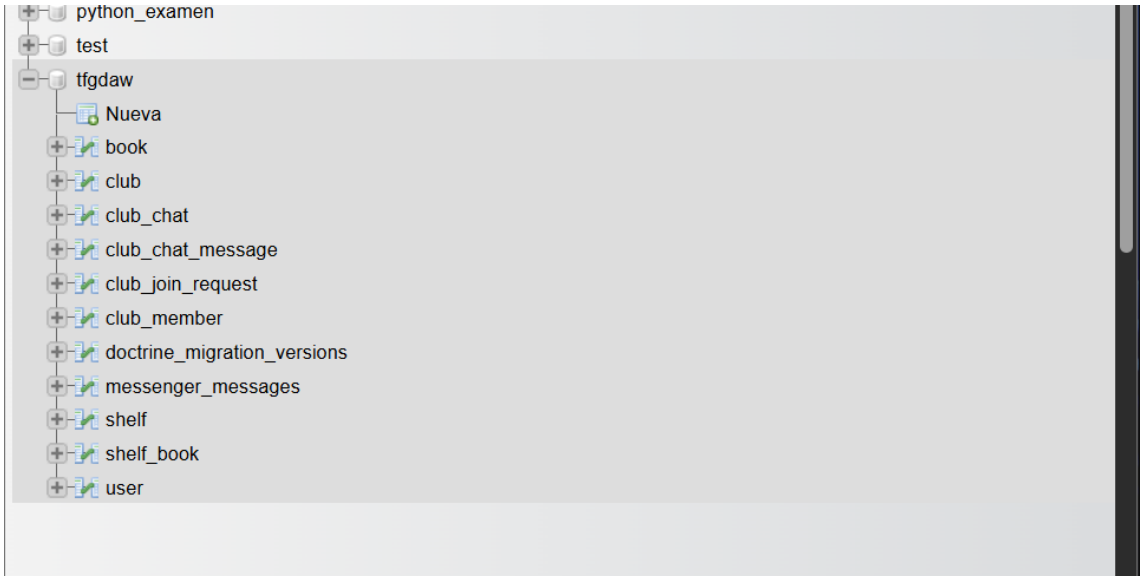
Chats

- crear chat
- listar mensajes
- enviar mensaje

👉 Esto es lo que convierte Symfony en un **backend para React**.

□ Cómo funciona (en humano)

1. El usuario hace login desde React
2. Symfony crea una **sesión**
3. El navegador guarda la cookie
4. Todas las peticiones siguientes ya están “logueadas”
5. Symfony sabe quién eres con getUser()


python_examen
test
tfgdaw
Nueva
book
club
club_chat
club_chat_message
club_join_request
club_member
doctrine_migration_versions
messenger_messages
shelf
shelf_book
user